WORKSHOP LAB

(Project Report)

Parts Counter Based on Blob Algorithm



Submitted by

AMAN KARKI BT24ECE070

SHRIJIT WARWATKAR BT24ECE071

OMKAR HURNE BT24ECE074

ROHAN PHANSE BT24ECE096

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

1 Introduction and Background

Counting small parts such as resistors, screws, nuts, bolts and SMD components is a frequent task in manufacturing, electronics workshops, inventory management and hobbyist workspaces. Manual counting is slow and error-prone; weight-based methods require precise scales and uniform parts; and industrial counting machines are often prohibitively expensive for small businesses and hobbyists. Computer vision offers a practical alternative: by capturing an image of parts placed on a contrasting surface and analyzing it algorithmically, a system can identify and count individual objects quickly and consistently. This approach reduces human error, provides repeatable results, and can be implemented with affordable, off-the-shelf components.

Blob detection is the core computer vision technique used in this project. In image processing, a "blob" is a connected region of pixels that share common characteristics such as intensity or color. Blob detection algorithms identify these regions, filter out noise and irrelevant detections, and return the set of valid blobs for counting. The general pipeline converts the captured image into a format suitable for analysis, applies preprocessing to enhance features and suppress noise, thresholds the image to separate foreground from background, and then performs connected-component analysis and filtering to produce the final count.

2 Problem Statement and Objectives

The primary goal is to design and implement a cost-effective, accurate and user-friendly automated parts counting system that uses computer vision and blob detection. The system should be accessible to hobbyists and small businesses while delivering accuracy comparable to more expensive industrial solutions. The main objectives are to produce a prototype that achieves at least 95% counting accuracy under controlled conditions, to keep hardware costs low by using affordable components, to implement robust blob detection and preprocessing routines that work under varying lighting and part types, and to document the process so others can reproduce and adapt the system.

Secondary objectives include improving counting speed through algorithmic and parameter optimizations, investigating methods to handle overlapping or touching parts, developing calibration procedures for different lighting conditions and part types, and establishing guidelines for optimal part placement and imaging. Success metrics include achieving the accuracy target, keeping processing time per count under five seconds on the chosen hardware, maintaining a low system cost, and ensuring consistent results over repeated runs.

3 System and Architecture Overview

The proposed system is organized into three cooperating subsystems:

1) Image acquisition
2) Image processing
3) User output

These subsystems operate together in a structured pipeline, where each stage performs a specific role and passes its results to the next stage. The overall design ensures that raw visual data is gradually refined into accurate and meaningful output for the user

3.1 Image Acquisition Subsystem

The image acquisition subsystem serves as the entry point of the system and is responsible for capturing high-quality images of the objects to be counted. It comprises a camera module, a controlled lighting arrangement, and a stable imaging platform. The camera captures the scene while the lighting setup ensures uniform illumination, thereby reducing shadows, reflections,

and uneven brightness that could degrade detection accuracy. A stable mounting platform maintains a fixed distance and angle between the camera and the objects, producing images with minimal variation; this consistency simplifies downstream processing and improves overall reliability. In short, this subsystem converts the physical arrangement of parts into a repeatable digital image suitable for analysis.

3.2 Image Processing Subsystem

The image processing subsystem forms the core of the system and performs the computational analysis required to identify and count objects. Once an image is acquired, it is passed to this subsystem where it undergoes a sequence of preprocessing and analysis steps. The image is first converted to grayscale to reduce data complexity while preserving intensity information. Denoising techniques are then applied to remove unwanted disturbances and smooth the image, which is essential to prevent false detections. Contrast enhancement methods, such as CLAHE, are used to make objects more distinguishable from the background, particularly in low-contrast situations. The enhanced image is converted to a binary representation through thresholding so that objects appear as connected foreground regions. Blob detection techniques are then applied to identify connected regions corresponding to individual objects; each detected region is analyzed and filtered to remove irrelevant areas such as noise, very small particles, or artifacts that do not meet predefined criteria. The subsystem may also retain intermediate results for debugging or performance evaluation. By the end of this stage, the system has identified and counted the objects present in the image.

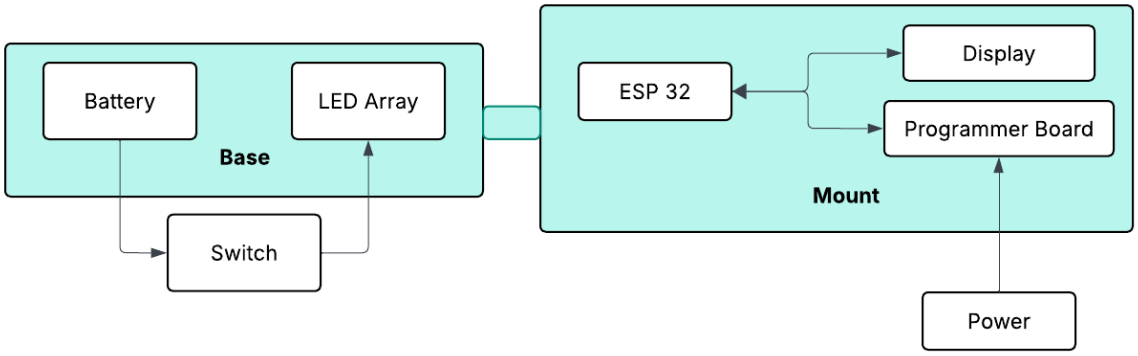
3.3 User Output Subsystem

The user output subsystem is responsible for presenting processed results in a clear and accessible manner. After image processing is complete, the final count and any additional statistical information are conveyed to the user through appropriate interfaces, which may include a local display, a serial monitor, or integration with a host application. Beyond simple display, the subsystem can support data export features to save results for further analysis or record-keeping. The emphasis of this stage is on clarity and usability so that users can interpret results quickly without needing to understand the internal processing details.

3.4 Data Flow and Operating Modes

The system follows a sequential data flow that begins with image capture and proceeds through preprocessing, detection, filtering, and result presentation. Each stage depends on the output of the previous one, creating a logical and efficient processing chain. To accommodate different use cases, the system supports multiple operating modes. In single-shot mode, a single image is captured and processed, which is suitable for quick counting tasks. Continuous mode enables real-time operation by processing images continuously for monitoring applications. Batch mode allows multiple stored images to be processed in sequence for offline analysis. A calibration mode is provided to fine-tune parameters such as threshold levels and detection criteria, ensuring optimal performance under varying lighting conditions and object types.

4 Block Diagram



5 Hardware Components and Specifications

The hardware includes mainly the following :

1) Processing unit
2) Lighting and imaging platform
3) Display

5.1 Processing Unit and Camera Options

A compact microcontroller camera module such as the ESP32-CAM provides a budget-friendly option for simple counting tasks. The ESP32-CAM integrates an OV2640 2MP camera and WiFi connectivity, and is suitable for low-resolution counting tasks where processing can be performed either on-device (limited) or by streaming images to a more capable host for processing. For higher accuracy and more advanced preprocessing, a small single-board computer or an external host can be used to run the full OpenCV stack; however, those platform details are omitted here per your instruction. The recommended camera for improved accuracy is a standalone camera module with a clear optical path and stable mounting. If using the ESP32-CAM, expect lower resolution and limited on-device processing; if streaming images to a host, ensure reliable connectivity and sufficient bandwidth.

Component	**Typical Details**
ESP32-CAM	OV2640 2MP camera, WiFi, limited GPIO, 5V power
Standalone camera module (example)	Higher resolution sensor recommended for improved accuracy
USB webcam (alternative)	Full HD 1080p, USB interface, autofocus (if available)

Table 1: Processing and camera options (focus on ESP32-CAM and generic camera modules)



Figure 1: Esp32 Cam Module

5.2 Lighting, Platform and Mount (3D-printed CAD)

Consistent, diffuse lighting and a stable mounting platform are combined into a single, integrated 3D-printed assembly to improve repeatability and simplify construction. The assembly houses diffused LED strips or a ring LED around the imaging aperture, provides a fixed, non-reflective imaging surface, and includes a rigid camera mount that fixes height and orientation relative to the mat. By integrating lighting, platform and mount into one CAD model, the design ensures a constant geometry between light sources, camera, and parts, which reduces shadows, minimizes reflections, and eliminates variation caused by manual repositioning. The imaging surface is a recessed matte tray within the 3D-printed assembly sized to accept a high-contrast mat (white for dark parts or black for light parts). The lighting channels are designed with diffuser slots and baffles to produce even illumination across the tray and to reduce specular highlights on reflective components. The camera mount is a keyed socket that accepts a standard camera module or small USB webcam and locks the module at a fixed focal distance; the socket can be adapted to different camera footprints with simple CAD variants. The assembly also includes cable routing channels and mounting points for a small power supply or battery compartment, enabling both bench and portable operation.



(a) Mount (b) Lighting and imaging platform

Figure 2: Overall 3D Printed Model

5.3 Display

The display subsystem presents the count and optional statistics in a compact, user-friendly format. For embedded deployments, a small OLED or LCD module is mounted on the front face of the 3D-printed assembly so that results are visible without an external host. The display shows the current count, a simple status indicator (ready, processing, error), and basic statistics such as average blob size or last-capture timestamp. When more detailed analysis or logging is required, the display can be complemented by a serial or network interface to export results to a host computer or mobile device.

The display mounting is integrated into the CAD model with a recessed pocket and screw bosses sized for common modules (for example, 0.96" I2C OLED or 1.3" SPI LCD). The design allows easy replacement of the display module and provides access to programming/debugging pins through a small removable panel. For clarity during development, include a simple on-screen legend and a single tactile button on the assembly to trigger captures or cycle display modes.



Figure 3: Display

5.4 Bill of Materials (BOM)

The BOM below focuses on the low-cost configuration centered on an ESP32-CAM or a generic camera module. Prices are approximate and vary by supplier and region.

Item	Quantity	Estimated Cost
ESP32-CAM module	1	740
USB programmer	1	220
OLED display	1	420
LED roll	1	323
9V battery	1	90
Total		**1793**

Table 2: Bill of materials

6 Software Architecture and Algorithms

The software is designed using a layered architecture to keep different responsibilities organized and easy to manage. At the top is the application layer, which handles user interaction and displays the final results. Beneath it is the processing layer, where all the core image analysis takes place, including detecting and filtering objects in the image. At the base is the library layer, which uses established tools such as OpenCV and NumPy when the system has sufficient computational capability. This separation of layers helps make the system more modular, maintainable, and scalable.

The main objective of the system is to detect and count objects by identifying connected regions, commonly referred to as blobs, in an image. To achieve accurate detection, the image must first go through several preprocessing steps. Initially, the image is converted to grayscale, which reduces computational complexity and removes unnecessary color information. A Gaussian blur is then applied to smooth out noise while preserving important edges. After that, contrast is enhanced using methods like CLAHE, which helps objects stand out more clearly from the background, especially in low-contrast situations. Finally, thresholding is applied to convert the image into a binary format, making it easier to separate objects from the background. Adaptive thresholding is useful when lighting conditions are uneven, while global thresholding works well in controlled environments.

Once preprocessing is complete, the system uses a blob detection algorithm to identify and analyze objects. A commonly used approach is OpenCV's `SimpleBlobDetector`, which works by applying multiple threshold levels to the image, grouping connected pixels into blobs, and calculating their positions and sizes. The detector then filters these blobs based on specific criteria to ensure only relevant objects are counted. For example, area-based filtering removes very small blobs that are likely noise and very large blobs that may be irrelevant. Shape-based filters such as circularity, convexity, and inertia ratio help distinguish actual objects from irregular shapes or distortions. Additionally, setting a minimum distance between blob centers prevents the same object from being detected multiple times.

The accuracy of the system depends heavily on properly tuning these detection parameters. Important parameters include the threshold range, minimum and maximum blob area, shape constraints, blob color (whether objects are light or dark), and the minimum distance between detected blobs. Since these values may need adjustment based on different use cases or environments, they are typically stored in a configuration file, such as a JSON file. This allows the system to reuse optimized settings without requiring repeated manual adjustments. By combining effective preprocessing with carefully tuned parameters, the system can reliably detect and count objects even when conditions such as lighting and noise vary.

7 Applications

The parts counter is suitable for small-scale inventory tasks, hobbyist component sorting, quick quality-control checks where counts of discrete items are needed, and educational demonstrations of computer vision concepts. In small manufacturing or repair shops, the system can accelerate routine counting tasks and reduce human error. For educational use, the project demonstrates practical applications of image preprocessing, connected-component analysis and parameter tuning.

8 Limitations and Future Improvements

This project has known limitations and a short, prioritized list of improvements to increase accuracy, robustness, and usability.

Limitations

- Overlapping or touching parts may be counted as a single blob.
- Reflective surfaces and non-uniform backgrounds can cause false detections.
- Low-cost camera modules have limited resolution and on-device processing capability.

Improvements

Software

- **Watershed segmentation:** Split touching/overlapping parts to improve counts.
- **Auto-threshold (Otsu):** Automate threshold selection to reduce manual tuning.
- **Multi-size counting:** Detect and classify multiple part sizes in one pass.
- **Batch tracking:** Log counts with timestamps and metadata for inventory use.

Hardware

- **Higher-resolution camera and improved lens:** Enhance detection of small parts and reduce edge distortion.
- **Adjustable mount:** Allow quick focal-distance changes for different part sizes.
- **WiFi integration:** Enable automatic export of counts to remote systems.

User Interface

- **Touchscreen and presets:** On-device parameter adjustment and saveable presets.
- **Visual feedback:** Display blob outlines and rejected regions for quick validation.

Roadmap (summary)

1. Implement auto-thresholding and configuration persistence.
2. Add improved segmentation (watershed) and multi-size counting.
3. Introduce UI presets and on-device visual feedback.
4. Offer optional hardware upgrades and WiFi export.

9 Conclusion

This document presents a practical, low-cost approach to automated parts counting using blob detection and affordable camera modules. With careful attention to lighting, background contrast, and parameter tuning, the system can achieve high accuracy for many common part types. The design emphasizes modularity so that detection algorithms, preprocessing steps and hardware choices can be adapted to specific needs. Removing platform-specific sections (Raspberry Pi and Arduino) narrows the focus to camera and microcontroller options suitable for budget deployments while leaving room for integration with more capable hosts when higher accuracy or advanced processing is required.

10 References and Resources

OpenCV documentation: https://opencv.org/.
OpenCV SimpleBlobDetector reference: https://docs.opencv.org/.
ESP32-CAM module information: vendor datasheets and community tutorials.

A Code :

```
#include "esp_camera.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ===== OLED =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define SDA_PIN 14
#define SCL_PIN 13

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== CAMERA PINS (AI Thinker ESP32-CAM) =====
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

// ===== TUNABLE PARAMETERS =====
int thresholdOffset = 40;
int minArea = 40;
int maxArea = 3000;

// Fixed ROI
int roi_x = 40, roi_y = 30, roi_w = 80, roi_h = 60;

// ===== RAM =====
#define MAX_STACK 800
uint8_t* bin = NULL;
int* stackBuf = NULL;
```

```
int clamp(int v, int lo, int hi) {
  return v < lo ? lo : (v > hi ? hi : v);
}

void showMessage(const char* msg, uint8_t textSize = 2) {
  display.clearDisplay();
  display.setTextSize(textSize);
  display.setTextColor(WHITE);
  display.setCursor(0, 0);
  display.println(msg);
  display.display();
}

bool initDisplay() {
  // Help weak I2C lines a little on ESP32-CAM
  pinMode(SDA_PIN, INPUT_PULLUP);
  pinMode(SCL_PIN, INPUT_PULLUP);

  Wire.begin(SDA_PIN, SCL_PIN);
  Wire.setClock(50000);   // slower = more reliable on ESP32-CAM
  delay(50);

  if (display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) return true;
  if (display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) return true;
  return false;
}

int floodFill(int start, int w, int h) {
  int size = 0;
  int top = 0;

  stackBuf[top++] = start;
  bin[start] = 0;

  while (top > 0) {
    int idx = stackBuf[--top];
    size++;

    int n1 = idx - 1;
    int n2 = idx + 1;
    int n3 = idx - w;
    int n4 = idx + w;

    if (n1 >= 0 && bin[n1] == 255 && top < MAX_STACK) {
      bin[n1] = 0;
      stackBuf[top++] = n1;
    }
    if (n2 < w * h && bin[n2] == 255 && top < MAX_STACK) {
      bin[n2] = 0;
      stackBuf[top++] = n2;
```

```
    }
    if (n3 >= 0 && bin[n3] == 255 && top < MAX_STACK) {
      bin[n3] = 0;
      stackBuf[top++] = n3;
    }
    if (n4 < w * h && bin[n4] == 255 && top < MAX_STACK) {
      bin[n4] = 0;
      stackBuf[top++] = n4;
    }
  }

  return size;
}

int countObjects(camera_fb_t* fb) {
  int w = fb->width;
  int h = fb->height;
  int n = w * h;

  if (!bin) bin = (uint8_t*)malloc(n);
  if (!stackBuf) stackBuf = (int*)malloc(MAX_STACK * sizeof(int));
  if (!bin || !stackBuf) return -1;

  long sum = 0;
  int countPix = 0;

  int x0 = clamp(roi_x, 0, w - 1);
  int y0 = clamp(roi_y, 0, h - 1);
  int x1 = clamp(roi_x + roi_w, 0, w);
  int y1 = clamp(roi_y + roi_h, 0, h);

  for (int y = y0; y < y1; y++) {
    for (int x = x0; x < x1; x++) {
      int i = y * w + x;
      sum += fb->buf[i];
      countPix++;
    }
  }

  if (countPix == 0) return 0;

  int mean = sum / countPix;
  int thr = clamp(mean - thresholdOffset, 20, 230);

  for (int y = 0; y < h; y++) {
    for (int x = 0; x < w; x++) {
      int i = y * w + x;
      if (x >= x0 && x < x1 && y >= y0 && y < y1 && fb->buf[i] < thr) {
        bin[i] = 255;
      } else {
        bin[i] = 0;
```

```
      }
    }
  }

  int count = 0;

  for (int y = y0; y < y1; y++) {
    for (int x = x0; x < x1; x++) {
      int i = y * w + x;
      if (bin[i] == 255) {
        int area = floodFill(i, w, h);
        if (area >= minArea && area <= maxArea) {
          count++;
        }
      }
    }
  }

  return count;
}

void setup() {
  delay(300);

  // 1) OLED first
  if (!initDisplay()) {
    while (1) {
      delay(1000);
    }
  }

  showMessage("OLED OK", 2);
  delay(700);

  // 2) Camera next
  camera_config_t config = {};   // important: zero-initialize

  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;

  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;

  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
```

```
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;

  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;

  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;

  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_GRAYSCALE;
  config.frame_size = FRAMESIZE_QQVGA;
  config.fb_count = 1;
  config.fb_location = psramFound() ? CAMERA_FB_IN_PSRAM : CAMERA_FB_IN_DRAM;
  config.grab_mode = CAMERA_GRAB_LATEST;

  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    showMessage("CAM FAIL", 2);
    while (1) {
      delay(1000);
    }
  }

  showMessage("READY", 2);
  delay(700);
}

void loop() {
  camera_fb_t* fb = esp_camera_fb_get();
  if (!fb) return;

  int count = countObjects(fb);
  esp_camera_fb_return(fb);

  display.clearDisplay();
  display.setTextColor(WHITE);
  display.setTextSize(3);
  display.setCursor(20, 20);
  display.print(count);
  display.display();

  delay(200);
}
```